

Reviewer Pack — Minimal Order of Frobenius Schur Indicators and Circuit Weig...

Entry 92f0ad093ddc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 14:55:42 UTC

Minimal Order of Frobenius Schur Indicators and Circuit Weights Correlation

Entry ID: 92f0ad093ddc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 14:55:42 UTC

1. Conjecture

Field A (mathematical branch): Frobenius Schur Indicator Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For any given Boolean circuit C with n inputs, the minimal order of Frobenius Schur indicators ($\text{min_order}(\text{FSInd}(C))$) is linearly correlated with the total weight of C , such that $\text{min_order}(\text{FSInd}(C)) = \Theta(\text{weight}(C))$.

Rationale (proposer's reasoning):

Frobenius Schur indicators are a tool from representation theory that can capture information about the structure of a group or algebra. If their minimal order correlates with circuit weights, it suggests a deeper connection between the algebraic and computational properties of circuits.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): da65acf09ca2f02b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit C with n inputs, if the Pearson correlation coefficient between $\text{min_order}(\text{FSInd}(C))$ and $\text{weight}(C)$ from 30 random seeds is ≥ 0.8 and no seed yields a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius Schur Indicator Theory" AND "Boolean circuit complexity" AND correlation - "minimal order Frobenius Schur indicators" AND "circuit weight" AND linear relationship - "Boolean circuit complexity" AND "Frobenius Schur indicator theory" AND $\min_order(FSInd(C)) = \theta(weight(C))$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        circuit = []
        for _ in range(2**n):
            gate = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(n)]
            circuit.append((gate, inputs))
        return circuit

    def compute_weight(circuit):
        weight = 0
        for gate, inputs in circuit:
            if gate == 'AND':
                weight += sum(inputs)
            elif gate == 'OR':
                weight += len(inputs) - sum(inputs)
        return weight

    def frobenius_schur_indicator(gate, inputs):
        if gate == 'AND':
            return Fraction(sum(inputs), 2**len(inputs))
        elif gate == 'OR':
            return Fraction(len(inputs) - sum(inputs), 2**len(inputs))

    def min_order(FSInds):
        return min(FSInd for FSInd in FSInds if FSInd != 0)
```

```

n = random.randint(5, 40)
circuit = generate_random_circuit(n)
weight = compute_weight(circuit)
FSInds = [frobenius_schur_indicator(gate, inputs) for gate, inputs in circuit]
min_order_FSInd = min_order(FSInds)

return {
    "metric_name": "min_order_FSInd_vs_weight",
    "metric_value": min_order_FSInd * weight,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] and res["metric_value"] < 0.5 for res in results):
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"low_correlation\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14254
2	propose	ollama_remote	glm4:latest	0	0	21851
3	preregistration	ollama_remote	glm4:latest	0	0	9040
4	novelty	ollama_remote	glm4:latest	0	0	8363
5	novelty	ollama_remote	glm4:latest	0	0	8661
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15862
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16263
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37300
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10800
10	judge	ollama_remote	glm4:latest	0	0	20039

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162432 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/92f0ad093ddc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/92f0ad093ddc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/92f0ad093ddc.tar.gz (if generated)